Homework 7: RNN

Xiaotong Hu, Zimo Qi

Question 1

(a) In `nextsup/nextdev`, the tag at position j is the uppercase of the **next** word: $t_j = \mathsf{upper}(w_{j+1})$, and $t_{n-1} = \texttt{EOS}$. In `possup/posdev`, all words are the same x, but tags follow a fixed period-4 pattern: t_j is $A, \_, B, \_$ repeating.

(b) In our previous HMM/CRF, the ϕ functions are written as$\phi_A(s, t), \phi_B(t, w_j)$. $\log \phi_B(t, w_j)$ depends only on the *current* word w_j and the tag t, and $\log \phi_A(s, t)$ depends only on adjacent tags. No local factor can take w_{j+1} as an argument. Although the forward–backward algorithm uses future tokens when computing posteriors $p(t_j \mid \mathbf{w})$ via β_j, this does not change the model's factorization for the score of a tag

$$\mathsf{score}(\mathbf{t} \mid \mathbf{w}) = \sum_i \log \phi_A(t_{i-1}, t_i) + \sum_i \log \phi_B(t_i, w_i)$$

which contains no term of the form as $\log \phi(t_j, w_{j+1})$. That is to say, in this form, if we take two sentences that agree on $(w_0, \ldots, w_j)$ but differ at w_{j+1}, then for any fixed tag sequence $\mathbf{t}$ the local contribution at position j is identical in both sentences. The model cannot give a deterministic rule such as $t_j = \mathsf{upper}(w_{j+1})$ for sentences in the `next` dataset. For `pos` dataset, it is the same.

We follo the INSTRUCTION to implement our `crftest` model which is a non-stationary CRF whose potentials depend on position and context. Concretely, we override $A\_at(j, \cdot)$ and $B\_at(j, \cdot)$ so that the emission log-potentials at position j are a sum of (i) a position feature depending on $j \bmod 4$ and (ii) a "next-word" feature depending on w_{j+1}, and then exponentiate to obtain the local potentials which directly encode both synthetic rules.

Empirically, we trained each model for 5 epochs and reported the best dev accuracy:

	`hmm`	`crf`	`crftest`
`next`	37.44%	43.38%	100.00%
`pos`	80.36%	80.36%	100.00%

The HMM and stationary CRF significantly underfit both synthetic datasets, while the non-stationary `crftest` achieves 100% accuracy.

Similar to the `crftest`, a biRNN-CRF uses **context-dependent** potentials $\phi_A(s, t, \mathbf{w}, j)$ and $\phi_B(t, w_j, \mathbf{w}, j)$ whose features can explicitly depend on the suffix (via the backward RNN state), so the local emission score at position j can not only directly encode the next word w_{j+1} and therefore represent the `next` labeling rule but also encode the repeating patterns. Please see the results in next question which will show the effectivenss of RNN.

(c) As shown in Table 1, a small hidden dimension d 2 is enough to encode the relevant contextual information. With $d = 0$, the model degenerates to a stationary CRF and underfits both toy tasks (40.29% on `next`, 80.36% on `pos`). Once a really small positive dimension (e.g. $d = 2$) is allowed, the biRNN–CRF can exploit great prefix and suffix representations and quickly reaches 100% dev accuracy on both datasets. Increasing d further (from 2 to 15) not only reduces the dev cross-entropy slightly but also reduce time to reach optimum 100%.

d	Acc (max) (%)	samples to reach optimum	Acc (max) (%)	samples to reach optimum
0	44.29	∞	80.36	∞
2	100.00	2600	100.00	2600
8	100.00	1200	100.00	1200
15	100.00	1200	100.00	800

Table 1: Dev performance of the biRNN–CRF with different d (trained with `--eval_interval 200 --max_steps 6000`). Left is on `next` dataset, right is on `pos`.

Question 2

(a) Table 2 compares the baseline HMM, the stationary CRF, and the default biRNN–CRF. In this default configuration, a biRNN does *not* improve dev performance. The HMM attains a very high dev accuracy (90.46%), while the basic CRF and the biRNN–CRF both achieve lower accuracies, with 85.70% and 78.50% respectively. The default biRNN–CRF here uses only a small lexicon, words-10, 10-dimensional embeddings, and a small RNN dimension ($d = 8$), and these choices limit the benefit of word representations – 10 dimension is not that informative. Later experiments in part (b) show that when we increase the embedding size and RNN dimensionality and tune the hyperparameters, the biRNN–CRF can substantially outperform the HMM.

Model	RNN dim d	Lexicon	Reg λ	Learning rate	Dev CE	Dev Acc (%)
HMM	0	–	–	–	10.81	90.46
basic CRF	0	–	0.0	0.01	0.18	94.24
biRNN-CRF	8	words-10	0.1	0.01	0.59	78.50

Table 2: Comparison of HMM, stationary CRF, and default biRNN-CRF(d8-word10) on *endev*

(b) From Table 3, increasing the CBOW lexicon size consistently helps!!! That is so great. Moving from `words-10` to `words-100` monotonically reduces dev cross-entropy (0.59 → 0.25) and raises dev accuracy (78.5% → 92.1%). Also, we found that the Google News embeddings are weaker than ordinary embeddings of the same dimensionality.

Lexicon	Dev CE	Dev Acc (%)
words-10	0.59	78.50
words-20	0.43	85.43
words-50	0.29	90.58
words-100	**0.25**	**92.11**
words-gs-10	0.64	77.26
words-gs-50	**0.40**	**86.76**

Table 3: Effect of lexicon on *endev* performance (fixed $d = 8$, reg $\lambda = 0.1$, $lr = 0.01$, batch size 30)

Table 4 shows that increasing the RNN dimensionality d also improves performance, as desired !!!! It is even when combined with a richer lexicon. With the tiny lexicon `words-10`, raising d from 4 to 8 already gives a large gain (68.8% → 78.5%). With the stronger `words-50` lexicon, further increasing d from 8 to 16 and 32 continues to reduce dev CE and increase accuracy (90.6% → 92.4% → 93.6%). They outperform the HMM baseline.

In Table 5, changing the learning rate mainly affects how aggressively the model updates, and here lr = 0.01 gives the best dev CE/Acc. This suggests that for this architecture, a moderate learning rate is

Model	RNN dim d	Lexicon	Dev CE	Dev Acc (%)
biRNN-CRF	4	words-10	0.88	68.84
biRNN-CRF	8	words-10	0.59	78.50
biRNN-CRF	32	words-10	**0.32**	**88.02**
biRNN-CRF	8	words-50	0.29	90.58
biRNN-CRF	16	words-50	0.24	92.35
biRNN-CRF	32	words-50	**0.19**	**93.55**

Table 4: Effect of RNN dimensionality d on *endev* (fixed lexicon, reg $\lambda = 0.1$, $lr = 0.01$, batch size 30)

enough to reach a good optimum, and pushing lr higher makes optimization skip out of optimum. We may talk more about the training process in (c)

RNN dim d	Lexicon	Learning rate	Dev CE	Dev Acc (%)
8	words-50	0.01	0.29	90.58
8	words-50	0.05	0.31	89.65
8	words-50	0.1	0.31	89.69

Table 5: Effect of learning rate on *endev* (fixed $d = 8$, lexicon, reg $\lambda = 0.1$, batch size 30)

Table 6 shows that varying the L2 regularization strength λ between 0 and 0.1 has almost no effect on dev cross-entropy or accuracy in this range. It means that regularization is not a bottleneck hyperparameter compared to others. We may try more choices if we have more time.

RNN dim d	Lexicon	Reg λ	Dev CE	Dev Acc (%)
8	words-50	0.00	0.29	90.58
8	words-50	0.01	0.29	90.58
8	words-50	0.10	0.29	90.58

Table 6: Effect of regularization strength on *endev* (fixed $d = 8$, lexicon, $lr = 0.01$, batch size 30)

(c) From Figure 1, we can observe that larger lexicons not only converge faster but also reach lower final cross-entropy, which is consistent with the previous results in Table 3. As for the learning-speed, the curves for high-dimensional lexicons are relatively smooth (`words-100` stays around a moderate value around 4), whereas smaller lexicons such as `words-10` show fluctuations. The first logged average learning speed is very high, the second drops, and later points gradually increase again. This is because the average learning speed is computed as a running mean over minibatches, which introduce an initially large gradient on the first minibatch followed by much smaller gradients on the second minibatch. Moreover, smaller lexicons tend to induce more variable gradients across minibatches, while larger lexicons yield more stable gradient norms and hence smoother training dynamics.

Figure 2 compares different RNN dimensions. On the dev cross-entropy side, both the `w10` and `w50` curves show a clear monotonic pattern. For a fixed lexicon, larger RNN dimensionality d consistently yields lower dev CE and thus better performance. Also, all `w10` curves lie above `w50` curves entirely, which agrees with our earlier observation that richer word embeddings help the biRNN–CRF converge to better optima. In the learning-speed panel, we again see that smaller models with fewer parameters are less stable, particularly visible for the $d = 4$, `w10` configuration.

Figure 3 visualizes how batch size, learning rate, and regularization coefficient influence both the learning process.

- Looking at the top panel in Figure 3, we observe that the three batch sizes mainly differ in the

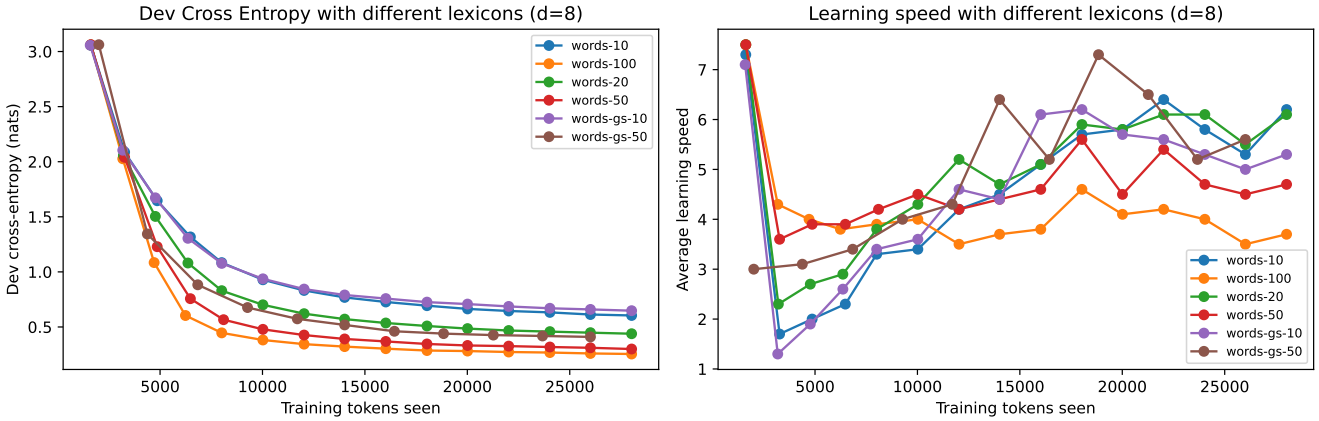

Figure 1: Dev cross-entropy (left) and learning speed (right) vs. training tokens for different lexicons (fixed $d = 8$, same hyperparameters).

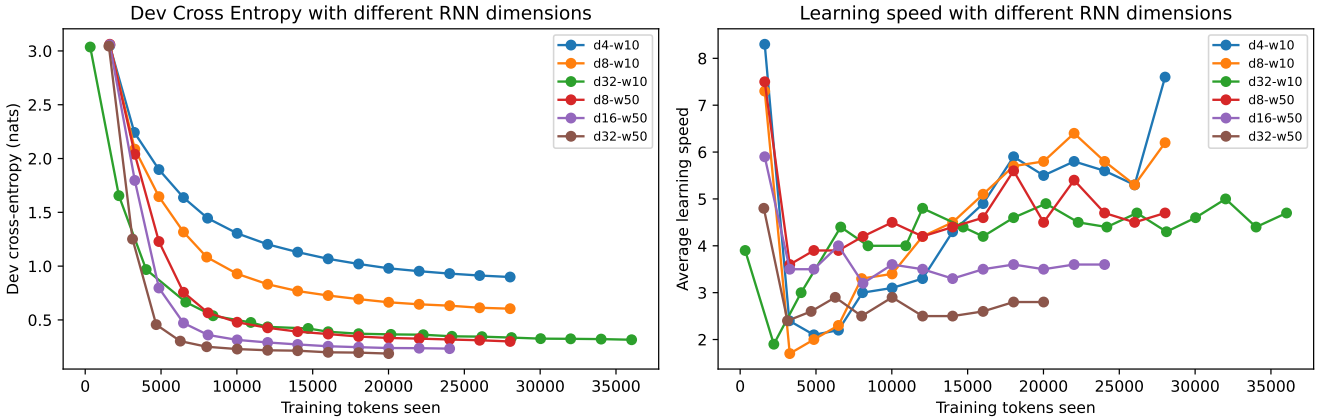

Figure 2: Dev cross-entropy (left) and learning speed (right) vs. training tokens for different RNN dimensions d.

very early stages of training. Small batches (like $b = 10$) produce a lower initial average learning speed, while larger batches ($b = 60$) are larger. However, as training proceeds and models approach a good solution, the gradients become similar across batch sizes, and the running-average definition of "learning speed" smooths out the early differences. As a result, the learning-speed curves for $b = 10$, $b = 30$, and $b = 60$ converge to almost the same level later in training. It indicates that batch size mainly affects the behavior at the beginning rather than the asymptotic convergence rate.

- As in the middle panel, increasing the learning rate from 0.01 to 0.05 or 0.1 significantly boosts the peak learning speed at the beginning (which is expected by the definition of learning speed) and causes CE to drop very quickly.
- Varying the regularization coefficient does not affect much.

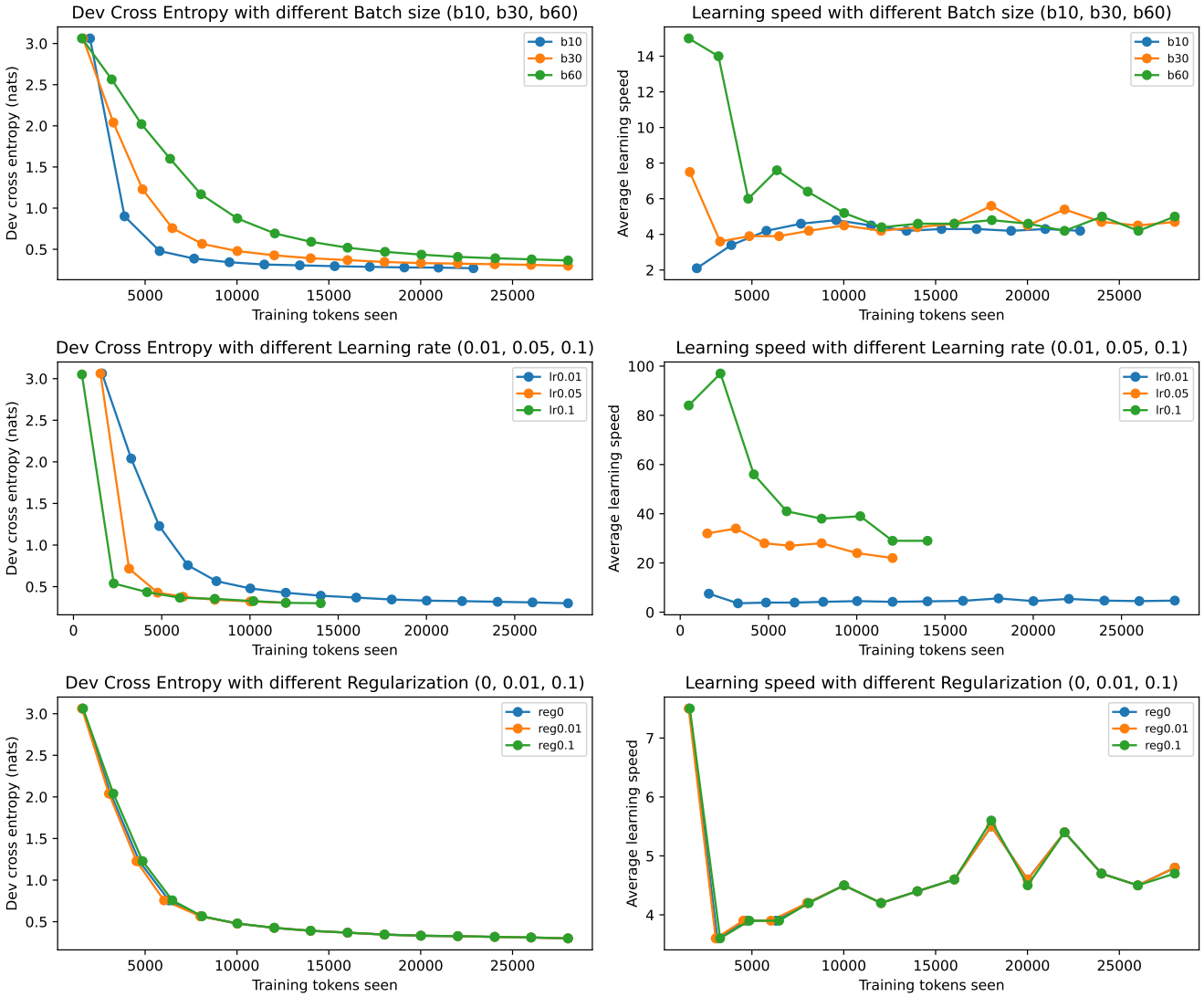

Figure 3: Dev cross-entropy (left column) and learning speed (right column) vs. training tokens for different batch sizes (top row), learning rates (middle row), and regularization strengths (bottom row).

(d) Table 7 compares the performance of the simple CRF and the biRNN–CRF on both the training set (`ensup`) and the dev set (`endev`). As expected, both models achieve substantially lower cross-entropy and higher accuracy on the training data than on the dev data, showing that they are able to fit the training set very well. The basic CRF typically attains even lower training loss than the BIRNN-CRF. But the train–dev gap can also be larger, indicating a stronger tendency to overfit.

Model	Dataset	Cross-entropy (nats)	Accuracy all (%)
Basic CRF	`ensup`	0.08	97.99
	`endev`	0.18	94.24
en-birnn-d8-w50	`ensup`	0.28	91.408
	`endev`	0.29	90.58

Table 7: Comparison of simple CRF and biRNN–CRF on training vs. dev data.

Question 3

(a) Table 8 compares the four biRNN–CRF models with full contextual features ($d = \texttt{rnn\_dim} = 8$) on `endev`. Using only one-hot word embeddings gives the best overall performance, lowest dev cross-entropy and highest overall and known-word accuracy. CBOW embeddings and `problex` features both help the biRNN–CRF reach good performance, but neither of them surpasses the simple one-hot baseline.

Features	Cross-entropy (nats)	Acc. all (%)
One-hot (no lexicon, no problex)	**0.189**	**94.16**
CBOW only (lexicon, no problex)	0.343	89.01
Problex only (no CBOW lexicon)	0.250	92.50
CBOW + problex	0.280	91.47

Table 8: biRNN–CRF ($d = 8$) on `endev` with different feature combinations.

(b) We turn off the RNN context by setting $\texttt{rnn\_dim} = 0$, then the number of trainable parameter in the model is zero. We fully rely on the information in embedding. Table 9 reports the results. All CBOW/problex configurations with $\texttt{rnn\_dim} = 0$ behave degenerately. They have no trainable parameters, their dev cross-entropy stays at 3.05 nats, and tagging accuracy is essentially random.

Features (no RNN context)	Cross-entropy (nats)	Acc. all (%)	Acc. known (%)	Acc. novel (%)
CBOW only	3.051	0.89	0.98	0.00
Problex only	3.051	0.89	0.98	0.05
CBOW + problex	3.051	0.89	0.98	0.00

Table 9: Embedding-only models ($\texttt{rnn\_dim} = 0$) on `endev`.

(c) Yes. From Table 8, with biRNN context, one-hot embeddings achieve the best accuracy higher than any other previous models Frequency-based features (`problex`) mainly help on known words (very high known accuracy) but less on novel words. Adding CBOW and problex together improves known-word performance but does not significantly boost novel-word accuracy. This suggests that these features mainly refine predictions on frequent patterns rather than dramatically improving generalization to unseen forms.

(d) We now analyze the accuracies bucketed by `known/seen/novel` breakdown. In the evaluation code, a token is *known* if its word type appears in `ensup`, *seen* if it does not appear in `ensup` but does have an entry in the model's vocab/lexicon (e.g., via CBOW embeddings), and *novel* if it is mapped to the OOV symbol.

Table 10 reports the bucketed accuracies for the four biRNN–CRF models with $d = 8$, like in (a). For the one-hot and problex-only configurations, the lexicon is built only from `ensup`, so the model's vocab coincides with the supervised training vocabulary. As a result, every in-vocab dev token is *known*, and

there are no *seen* tokens. In this setting, one-hot+biRNN achieves the highest overall accuracy (94.16%) and the best performance on both known (96.94%) and novel (65.37%) tokens, while problex-only reaches **similar** known-word accuracy (96.32%) but is clearly weaker on novel words (52.89%). This suggests that **both one-hot identity features and problex features provide informative representations for distinguishing tokens**, since every token is associated with its own feature vector. However, the problex model learns to rely more heavily on frequency-based information—which is highly predictive on the supervised training distribution from which these features are constructed—and this reliance makes the generalization weaker than that of the one-hot model.

One the other hand, the CBOW-based models expand the vocab adding word types from `endev` that are absent from `ensup`. For the CBOW-only model, the seen-word accuracy is high (78.05%), better than novel-word accuracy (64.45%). This suggests that **CBOW embeddings+biRNN context provides a strong signal for tagging words that were never labeled but do have embeddings** (in-domain generalization). However, when we combine CBOW and frequency-based (`problex`) features, the seen-word accuracy drops sharply to 40.55%, even below the novel-word accuracy (51.13%). This indicates that the additional frequency-like features, while helpful for known words (raising known accuracy from 90.79% to 95.82%), will mislead the model on dev-only words. Similar to one-hot/problex comparison, the model learns to **rely heavily on problex features**, overriding the useful CBOW information and hurting (out-domain) generalization.

Features	Acc. all (%)	Acc. known (%)	Acc. seen (%)	Acc. novel (%)
One-hot + biRNN	**94.16**	**96.94**	–	**65.37**
CBOW + biRNN	89.01	90.79	**78.05**	64.45
Problex only + biRNN	92.50	96.32	–	52.89
CBOW + problex + biRNN	91.47	95.82	40.55	51.12

Table 10: Bucketed dev accuracies for biRNN–CRF with $d = 8$ under different feature configurations.

Question 4

Improvement 1: Fine-tuned Embeddings

We implemented one of the suggested extensions: we *fine–tune the word embeddings* instead of keeping the lexicon fixed. Concretely, in the `ConditionalRandomFieldNeural` class we add an option `tune_lexicon`. When this flag is on (triggered by the `--awesome` command-line option), the lexicon matrix $E \in \mathbb{R}^{V \times e}$ is wrapped as an `nn.Parameter`, so SGD updates both E and the CRF parameters. When the flag is off, E is registered as a fixed buffer, which reproduces the originaln biRNN–CRF model.

We trained both models on `ensup` and evaluated on `endev`, using the lexicon `lexicons/words-10.txt` and hyperparameters $rnn\_dim = 8$, batch size 30, learning rate 0.01, L_2 regularization 0.1, and `eval_interval` = 2000. Table 11 summarizes the results.

Model	Overall Acc.	Known Acc.	Novel Acc.	Cross-Entropy (nats)
biRNN–CRF (baseline)	79.53%	79.04%	73.36%	0.5583
biRNN–CRF + tuned E	**92.23%**	**94.53%**	47.06%	**0.2365**

Table 11: Dev set performance with and without fine–tuning the word embeddings (`endev`)

What we observed from the results: Fine–tuning the embeddings substantially improves the dev cross–entropy (from 0.5583 to 0.2365 nats) and raises overall tagging accuracy (from 79.53% to 92.23%). Most of the gain comes from *known* words: known accuracy jumps from 79.04% to 94.53%. In contrast,

novel–word accuracy actually drops from 73.36% to 47.06%. This is consistent with the fact that we only update embeddings for vocabulary items that appear in training: the model becomes much better calibrated on known words, but the adapted embedding space may generalize less well to truly unseen word types. Overall, however, the large reduction in cross–entropy and the improvement in total accuracy indicate that the fine–tuned model is considerably better on this dev set.

What we have added and why it is helpful:

1. **Fine–tuning the lexicon embeddings:** In the baseline model, the CBOW embeddings from `words-10.txt` are fixed. They encode mostly semantic similarity and are not optimized for POS tagging. By turning E into trainable parameters, we allow the CRF to adapt the embedding space to the tagging objective: words that share syntactic behavior can move closer together, and dimensions that are irrelevant for POS can be deemphasized. This behaves like a task–specific retraining of the lexicon Empirically, this yields a large boost on known tokens and reduces dev cross–entropy, which suggests better calibration of the model's probabilities.

2. **Tradeoff between known and novel words:** A side–effect of aggressively adapting the embedding space is that representations may overfit to the supervised vocabulary. Our numbers show a clear gain on known words (which dominate the dev set) but a drop on novel words. This illustrates a general phenomenon: tuning embeddings improves performance where we have supervision, at the cost of potentially weaker generalization to unseen types.

Improvement 2: Fine-tuned Embeddings + Simpler FF architecture

Besides fine–tuned the embeddings, we also changed the neural architecture that defines the CRF potentials. In the original biRNN–CRF, the feed–forward network had to be run separately for each tag unigram or tag bigram at position j to compute the corresponding log–potentials. In our newly improved model (we call it `ConditionalRandomFieldNeuralPlus`), we instead run the feed–forward network **once** per position j on a context vector and then use a linear layer to produce all transition or emission scores from the same hidden representation. Concretely, for transitions we construct $c_j^A = [1; h_{j-1}; h'_j]$ and map it to a hidden vector $z_j^A = \tanh(W_A c_j^A + b_A)$, from which we predict all k^2 transition log–potentials by a linear map. For emissions we similarly use $c_j^B = [1; h_{j-1}; w_j; h'_j]$ and a second feed–forward layer to obtain all k emission log–potentials for the current word w_j from a shared hidden vector z_j^B.

We trained this "plus" model on `ensup` and evaluated on `endev` under the same hyperparameters as before ($rnn\_dim = 8$, batch size 30, learning rate 0.01, L_2 regularization 0.1, `eval_interval` = 2000, and lexicon `lexicons/words-10.txt`). Table 12 compares the fine–tuned model from Improvement 1 with the new architecture.

Model	Overall Acc.	Known Acc.	Novel Acc.	Cross-Entropy (nats)
biRNN–CRF (baseline)	79.53%	79.04%	73.36%	0.5583
biRNN–CRF + tuned E	92.23%	94.53%	47.06%	0.2365
biRNN–CRF + tuned E + simple FF	**92.50%**	**94.90%**	**70.68%**	**0.2186**

Table 12: Dev set performance of the baseline model, fine–tuned model with and without the simpler feed–forward architecture (`endev`).

For completeness, Table 13 reports the training times for all three models on `ensup`.

What we observed from the results: Relative to the fine–tuned improvement, the simpler FF architecture gives a small but consistent improvement in overall accuracy (from 92.23% to 92.50%) and a further reduction in dev cross–entropy (from 0.2365 to 0.2186 nats). Known–word accuracy also increases

Model	Total training time
biRNN–CRF (baseline, fixed E)	1:24:52
biRNN–CRF + tuned E	1:06:55
biRNN–CRF + tuned E + simple FF	**0:55:36**

Table 13: Training time for the different models on `ensup`.

slightly, from 94.53% to about 94.90%. The largest change is on novel words: accuracy jumps from 47.06% to 70.68%, largely recovering the drop we saw when we first fine–tuned the embeddings. Thus, the new architecture keeps most of the gains from Improvement 1 on known tokens while substantially improving performance on truly unseen word types.

In addition, the new model is **more efficient**. As Table 13 shows, the fine–tuned baseline already trains faster than the original CRF, but the "plus" architecture reduces training time even further, from 1:06:55 to 0:55:36 (about a 15–20% speedup). So we get slightly better accuracy and noticeably better novel–word performance, and a shorter training time.

What we have added and why it is helpful:

1. **Shared FF hidden layer for all potentials:** By running a single feed–forward network per position and reusing its hidden layer to predict all transition or emission scores, we force the model to encode the local context into a compact representation z_j that must support all tags. This encourages parameter sharing across tags and tag bigrams and acts as a form of regularization, which can reduce overfitting relative to having many separate FF computations.

2. **Better interaction with tuned embeddings:** Since the word embeddings and biRNN states h_{j-1}, h'_j are already adapted to the POS task (Improvement 1), the new architecture only needs to learn a smooth mapping from these task–specific context vectors to the CRF potentials. In our experiments this yields slightly better calibration (lower cross–entropy) and noticeably better accuracy on novel words, while preserving the high accuracy on known tokens and reducing training time. **For this homework, these improvements justify using the combined model `birnn_crf_awesome.pkl` as our strongest "awesome" system.**